RAMDEOBABA UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

[B.Tech. in Computer Science and Engineering MACHINE LEARNING LAB]

ON

DEVELOPMENT OF REVIEWGUARD: AN INTELLIGENT FAKE PRODUCT REVIEW DETECTION SYSTEM

Submitted by

Prashant Raghuwanshi (A4-27)

Swyam Gupta (A4-28)

Kush Jajoo (A4-30)

Under the Guidance of

Prof. Bharti Kungwani

Prof. Shubhangi Tirpude

Faculty Mentor

Academic Year: 2026–2027

[Semester V | Course Code: 24CS01PR0505]

RAMDEOBABA UNIVERSITY, NAGPUR

Department of Computer Science and Engineering

RAMDEOBABA UNIVERSITY, NAGPUR

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Project Report on **"REVIEWGUARD: INTELLIGENT FAKE PRODUCT REVIEW DETECTION SYSTEM"** is a bonafide work of Prashant Raghuwanshi, Swyam Gupta, Kush Jajoo submitted to Ramdeobaba University, Nagpur in partial fulfilment of the award of a Degree of Bachelor of Technology (B.Tech), in Computer Science and Engineering. It has been carried out at the Department of Computer Science and Engineering, Ramdeobaba University, Nagpur during the academic year 2026-27.

Date: [30/10/2026]

Place: Nagpur

Prof. Bharti Kungwani
Prof. Shubhangi Tirpude
Project Guide
Department of Computer Science & Engineering

Dr. Pravin Sonsare
Associate Head
Department of Computer Science & Engineering

Dr. P. D. Adane
Dean SoCSE

STUDENT DECLARATION

We hereby declare that the project report entitled "REVIEWGUARD: INTELLIGENT FAKE PRODUCT REVIEW DETECTION SYSTEM" is submitted as part of the requirements for the B.Tech programme at Ramdeobaba University.

The work presented in this report is prepared by the undersigned students under the guidance of the project supervisor. All external sources, datasets, software tools, figures and technical material used in the project have been appropriately acknowledged. The experimental results reported in this document are those obtained from the project's own evaluation on the stated dataset and test split.

Prashant Raghuwanshi — Roll No. A4-27
Swyam Gupta — Roll No. A4-28
Kush Jajoo — Roll No. A4-30

APPROVAL SHEET

This report entitled "**REVIEWGUARD: INTELLIGENT FAKE PRODUCT REVIEW DETECTION SYSTEM**" by **Prashant Raghuwanshi, Swyam Gupta, Kush Jajoo** is approved for the degree of Bachelor of Technology (B.Tech).

Dr. Pravin Sonsare
Department of Computer Science & Engineering

Prof. Bharti Kungwani
Prof. Shubhangi Tirpude
Project Guide

Dr. P. D. Adane
Dean SoCSE

ACKNOWLEDGEMENT

We express our sincere gratitude to Ramdeobaba University and the School of Computer Science and Engineering for providing the academic environment and facilities required to undertake this project. We are grateful to our project guide, Prof. Bharti Kungwani , Prof. Shubhangi Tirpude, for guidance, technical suggestions and feedback throughout the development of ReviewGuard.

We also acknowledge the open-source communities behind Python, scikit-learn and Flask, and the authors of the fake-review dataset used in this work, whose publicly available resources made this study possible. We thank the Project In-Charge, Associate Head and Dean, School of Computer Science and Engineering, for their support.

Finally, we acknowledge the support of our classmates, laboratory staff, friends and family members who contributed directly or indirectly to the completion of this work.

ABSTRACT

Online product reviews strongly influence purchasing decisions, which has created an incentive for astroturfing: the use of automated bots, text-generation models and incentivized writers to post fake reviews that inflate weak products or defame competitors. Human readers detect such reviews only slightly better than chance, so scalable automated detection is required. This project develops ReviewGuard, a lightweight system that classifies a product review as an original human-written review (OR) or a computer-generated synthetic review (CG).

ReviewGuard is trained on a corpus of 40,432 reviews drawn from ten consumer-product categories; after removing 12 exact duplicates, 40,420 reviews remain, with 20,215 original and 20,205 computer-generated samples. Text is sanitized by a five-step deterministic pipeline, converted into a 50,000-dimensional TF-IDF vector of unigrams and bigrams, and classified by an L2-regularized Logistic Regression model. Instead of a single 0.50 cut-off, a three-tier calibration rule labels a review Genuine ($P < 0.45$), Uncertain ($0.45 \leq P \leq 0.55$) or Fake/Suspicious ($P > 0.55$), so that borderline cases can be routed for manual audit.

On an 80/20 stratified split (32,336 training and 8,084 test samples), Logistic Regression achieved an accuracy of 89.19%, an F1-score of 89.16% and an ROC-AUC of 0.954, outperforming Multinomial Naive Bayes, LinearSVC and Random Forest in the comparison, with a model size of 2.8 MB and an inference time of about 24 ms on a CPU. The model is served through a Flask REST microservice (/api/predict and /api/health) with a responsive HTML5/CSS3/JavaScript interface and is deployed on Render using Gunicorn. Current limitations are the absence of reviewer-account metadata, limited handling of sarcasm and context, and susceptibility to fluent text from modern large language models; transformer models, graph neural networks, explainable AI and a browser extension are identified as future work.

Keywords: Fake review detection, opinion spam, NLP, TF-IDF, Logistic Regression, probability calibration, Flask, Render.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Certificate	2
2	Student Declaration	3
3	Approval Sheet	4
4	Acknowledgement	5
5	Abstract	6
6	List of Figures	8
7	List of Tables	9
8	List of Abbreviations	10
9	Chapter 1: Introduction	11
10	Chapter 2: Literature Review and Research Gap	14
11	Chapter 3: Proposed System and Methodology	18
12	Chapter 4: System Design and Implementation	22
13	Chapter 5: Results and Discussion	26
14	Chapter 6: Conclusion and Future Scope	31
15	References/Bibliography	33
16	Annexure A: Publications and Intellectual Property	34
17	Annexure B: Student Contribution Statement	35
18	Annexure C: Project Output	36

LIST OF FIGURES

Figure 3.1: Proposed ReviewGuard classification workflow

Figure 4.1: System architecture and data flow

Figure 4.2: ReviewGuard web interface – home screen

Figure 4.3: ReviewGuard web interface – prediction result

Figure 5.1: Category distribution of the 40,420 reviews

Figure 5.2: Model comparison – accuracy, F1-score and ROC-AUC

Figure 5.3: Confusion matrix of ReviewGuard on the test set

LIST OF TABLES

Table 2.1: Comparative literature matrix

Table 3.1: Text sanitization pipeline (clean_text)

Table 3.2: Dataset statistics after preprocessing

Table 4.1: Technology stack

Table 5.1: Category-wise distribution of reviews

Table 5.2: Quantitative comparison of classifiers

Table 5.3: Confusion-matrix-derived metrics for ReviewGuard

LIST OF ABBREVIATIONS

- AI — Artificial Intelligence
- API — Application Programming Interface
- AUC — Area Under the Curve
- CG — Computer-Generated (synthetic) review
- CI/CD — Continuous Integration / Continuous Deployment
- CPU — Central Processing Unit
- CSR — Compressed Sparse Row
- GCN — Graph Convolutional Network
- GNN — Graph Neural Network
- IDF — Inverse Document Frequency
- JSON — JavaScript Object Notation
- LIWC — Linguistic Inquiry and Word Count
- LLM — Large Language Model
- LR — Logistic Regression
- ML — Machine Learning
- MNB — Multinomial Naive Bayes
- NLP — Natural Language Processing
- OR — Original (human-written) review
- REST — Representational State Transfer
- ROC — Receiver Operating Characteristic
- SVM — Support Vector Machine
- TF — Term Frequency
- WSGI — Web Server Gateway Interface
- XAI — Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

Background

Problem Statement

Aim

Objectives

Scope

Organization of Report

CHAPTER 1: INTRODUCTION

1.1 Background

In the digital economy, consumer decisions are guided largely by peer opinions and crowd-sourced ratings. Industry surveys frequently report that a large majority of online shoppers read reviews before buying, and studies of review platforms have linked a one-star rating improvement to a 5% to 9% rise in revenue for the merchant concerned [9]. This commercial weight has encouraged astroturfing — the deceptive practice of commissioning bots, text-generation algorithms and incentivized click-farms to author synthetic reviews.

Such counterfeit reviews artificially boost substandard products or defame competing merchants, which erodes consumer trust in marketplaces and distorts fair competition. Language models have made the problem harder, because generated text can be fluent and varied, and ordinary readers find it difficult to tell it apart from genuine opinions [3].

1.2 Problem Statement

Human inspection is unreliable for detecting deceptive reviews: judges perform only slightly better than chance, partly because of cognitive biases such as confirmation bias [2]. The technical challenge is therefore to build an automated and scalable Natural Language Processing (NLP) and Machine Learning (ML) system that (i) discriminates between Original Human Reviews (OR) and Computer-Generated synthetic reviews (CG); (ii) responds within a strict latency bound of 50 ms on commodity CPU hardware, without requiring costly GPU infrastructure; and (iii) reports calibrated probabilistic confidence together with an explicit uncertainty buffer rather than an arbitrary binary cut-off.

1.3 Aim

To develop ReviewGuard, an intelligent fake product review detection system that classifies review text as genuine, uncertain or fake using TF-IDF features and Logistic Regression, and makes the result available to users through a deployed web application and REST API.

1.4 Objectives

The objectives are to: (i) prepare and sanitize a multi-category corpus of original and computer-generated product reviews; (ii) represent review text with TF-IDF unigram and bigram features; (iii) train and compare Multinomial Naive Bayes, LinearSVC, Random Forest and Logistic Regression classifiers on an identical stratified split; (iv) introduce a three-tier calibrated decision rule that exposes uncertainty; (v) evaluate the selected model using accuracy, precision, recall, F1-score, ROC-AUC, inference time and model size; and (vi) deploy the model as a decoupled Flask microservice with a responsive web interface on a cloud platform.

1.5 Scope

The system covers text-only classification of English product reviews using a corpus of 40,432 reviews from ten consumer domains. Deliverables are the text-sanitization and vectorization pipeline, the trained Logistic Regression classifier, the three-tier decision engine, and a Flask REST service (/api/predict and /api/health) with a Vanilla HTML5/CSS3/JavaScript interface deployed on Render. Reviewer-account metadata (account age, verified-purchase status, posting frequency, IP clusters), graph-based collusion detection and transformer-based models are outside the current scope and are listed as future work. ReviewGuard is a decision-support tool; borderline cases are intended for manual audit rather than automatic removal.

1.6 Organization of the Report

Chapter 1 introduces the problem, objectives and scope. Chapter 2 reviews representative literature and identifies the research gap. Chapter 3 describes the dataset, preprocessing, feature extraction, classifier and calibration rule. Chapter 4 documents the system architecture, technology stack and cloud deployment. Chapter 5 presents the results, model comparison and error analysis. Chapter 6 concludes the work and outlines future scope.

CHAPTER 2

INTRODUCTION

Opinion Spam and Metadata-Based Detection

Psycholinguistic and N-gram Approaches

Synthetic Review Generation and Detection

Deep Learning Approaches

Graph-Based Fraud Detection

Research Gap

Summary of Representative Literature

CHAPTER 2: LITERATURE REVIEW AND RESEARCH GAP

2.1 Introduction

Deceptive review detection has progressed from rule-based and metadata-driven filters to statistical learning, psycholinguistic analysis, deep neural networks and graph-based methods. This chapter reviews five representative studies, one for each of these directions, and positions ReviewGuard relative to them.

2.2 Opinion Spam and Metadata-Based Detection

Jindal and Liu [1] were among the first to study opinion spam at scale. They analysed millions of Amazon reviews and grouped spam into three types: untruthful reviews, reviews that target only the brand, and non-reviews such as advertisements. Using classifiers such as Logistic Regression, they showed that duplicate and near-duplicate reviews are widespread and that signals from review and reviewer behaviour, such as rating deviation and reviewer activity, are valuable when text alone is insufficient. Their limitation for the present problem is that a newly generated, non-duplicate synthetic review has no duplicate to match.

2.3 Psycholinguistic and N-gram Approaches

Ott et al. [2] built a gold-standard dataset of deceptive hotel reviews collected through Amazon Mechanical Turk and combined n-gram features with psycholinguistic categories from Linguistic Inquiry and Word Count (LIWC). They observed that deceptive writers use language differently from truthful ones, and a linear Support Vector Machine reached about 89.8% accuracy. The study is limited to a single domain (hotels), and its features do not transfer automatically to the broad product categories handled by ReviewGuard, but it established n-gram linear classifiers as a strong and interpretable baseline.

2.4 Synthetic Review Generation and Detection

Salminen et al. [3] addressed both the creation and the detection of fake reviews of online products. They experimented with two language models, ULMFiT and GPT-2, to generate fake reviews from an Amazon e-commerce dataset and used the better model, GPT-2, to build a dataset for the detection task. Their results show that detecting fake reviews is difficult for humans but that machine classifiers are effective, so that,

in their words, machines can fight machines. This dataset of original and computer-generated reviews across product categories is the direct benchmark for ReviewGuard.

2.5 Deep Learning Approaches

Hájek and Barushka [4] compared traditional machine-learning methods such as Naive Bayes, support vector machines and decision trees with deep neural networks, including convolutional and long short-term memory networks, for detecting positive and negative fake consumer reviews. They reported that the deep networks significantly outperformed the traditional methods in accuracy. Deep models therefore offer accuracy gains, but they require more training and serving resources than linear models, which is a trade-off that matters for a CPU-hosted service with a latency budget; ReviewGuard measures this trade-off for its own classical baselines in Chapter 5.

2.6 Graph-Based Fraud Detection

Dou et al. [5] moved from text-only analysis to graph-based fraud detection. They observed that fraudsters disguise themselves through feature camouflage and relation camouflage, and proposed CARE-GNN, a camouflage-resistant graph neural network that selects informative neighbours in a multi-relation graph using a label-aware similarity measure and reinforcement learning. On two real-world fraud datasets it outperformed other graph neural networks and fraud detectors. Graph methods can expose coordinated groups whose individual reviews look natural, but they need access to the complete reviewer–product history, which a text-only service does not have. This work forms the basis of ReviewGuard's graph-based future roadmap.

2.7 Research Gap

Metadata and graph methods require platform-level behavioural data, psycholinguistic studies were limited to single domains, and deep models increase serving cost. The practical gap addressed by ReviewGuard is a text-only detector that is trained on a multi-category corpus, runs within a tight CPU latency budget, has a very small model footprint, and reports calibrated probabilities with an explicit uncertainty band instead of a hard binary decision, together with a deployed, publicly accessible interface.

2.8 Summary of Representative Literature

Table 2.1 compares the reviewed studies with ReviewGuard. Reported results are taken from the cited papers where a directly comparable figure exists; otherwise the qualitative finding is stated, because the studies use different datasets and metrics and are not strictly comparable.

Table 2.1: Comparative literature matrix

Study	Year	Focus / domain	Primary models	Feature space	Reported result	Major limitation
ReviewGuard (ours)	2026	Multi-category retail	Calibrated Logistic Regression	TF-IDF (1,2)-grams, 3-tier thresholds	89.19% accuracy on own test split	Text-only; no account metadata
Salminen et al. [3]	2022	Amazon products; GPT-2 reviews	Language models for generation; ML classifiers for detection	Text features	Machine classifiers effective; humans struggle	Depends on generator used to create the data
Dou et al. [5]	2020	Fraud graphs	CARE-GNN	Multi-relation graph	Outperforms GNN baselines on two datasets	Needs full relational graph
Hájek & Barushka [4]	2019	Consumer reviews	CNN, LSTM vs. Naive Bayes, SVM, decision trees	Word representations	Deep networks outperform traditional methods	Higher model complexity

CHAPTER 3

Proposed Workflow

Dataset Description

Data Cleaning and Preprocessing

Feature Extraction using TF-IDF

Classification using Logistic Regression

Three-Tier Decision Calibration

Evaluation Plan

CHAPTER 3: PROPOSED SYSTEM AND METHODOLOGY

3.1 Proposed Workflow

ReviewGuard follows an end-to-end, decoupled pipeline. A raw, unstructured review is first sanitized by a deterministic function, then converted into a sparse TF-IDF vector of unigrams and bigrams with 50,000 dimensions. An L2-regularized Logistic Regression model produces a continuous logit score, which a bounded sigmoid converts into a probability that the review is synthetic. A three-tier calibration engine finally maps the probability to a Genuine, Uncertain or Fake/Suspicious label displayed with green, amber or red badges.

Figure 3.1: Proposed ReviewGuard classification workflow

3.2 Dataset Description

The empirical corpus contains 40,432 product reviews from ten consumer domains: Kindle Store, Books, Pet Supplies, Home & Kitchen, Electronics, Sports & Outdoors, Tools & Home Improvement, Clothing & Shoes, Toys & Games, and Movies & TV. Each review is labelled either OR (original, human-authored) or CG (computer-generated). Because the reviews span multiple domains, the classifier cannot rely on the vocabulary of a single category.

3.3 Data Cleaning and Preprocessing

Real-world reviews contain web-scraping artifacts, promotional links, irregular capitalization and emoji noise. The clean_text function applies the five operations in Table 3.1 in the order shown.

Table 3.1: Text sanitization pipeline (clean_text)

Step	Operation	Implementation	Technical justification
1	Lowercasing	str(text).lower()	Normalizes the vocabulary so that “GREAT” and “great” share one feature.
2	HTML stripping	re.sub(r"<.*?>", " ", text)	Removes scraping tags such as , and .
3	URL removal	re.sub(r"http\S+|www\S+", " ", text)	Removes hyperlink tokens injected by spam bots that distort word frequencies.
4	Special-character filter	re.sub(r"[^a-z0-9\s]", " ", text)	Strips emojis, non-ASCII glyphs and punctuation artifacts.
5	Whitespace normalization	re.sub(r"\s+", " ", text).strip()	Collapses multiple spaces, carriage returns and tabs into one space.

After cleaning, 12 exact duplicate rows were removed, leaving 40,420 reviews with an almost perfectly balanced class distribution (Table 3.2).

Table 3.2: Dataset statistics after preprocessing

Item	Value
Raw corpus entries	40,432
Exact duplicate rows dropped	12
Final processed rows	40,420
OR (original / human-authored)	20,215 (50.01%)
CG (computer-generated / synthetic)	20,205 (49.99%)

3.4 Feature Extraction using TF-IDF

Cleaned text is converted into numerical features with Term Frequency–Inverse Document Frequency (TF-IDF) over unigrams and bigrams, limited to 50,000 dimensions. For a term t in document d within a corpus D of N documents, the term frequency, smoothed inverse document frequency and their product are:

$$TF(t, d) = f_{t,d} \,/\, \Sigma_{t' \in d}\, f_{t',d}$$

$$IDF(t) = \ln(\,(1 + N) \,/\, (1 + DF(t))\,) + 1$$

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Each document vector v is then scaled to unit Euclidean length so that long and short reviews are comparable:

$$x = v \,/\, \|v\|_2, \quad where \quad \|v\|_2 = (\,\Sigma_{j=1}^{M}\, v_j^2\,)^{1/2}$$

The result is stored as a sparse Compressed Sparse Row (CSR) matrix, since most of the 50,000 features are zero for any single review. Because the L2 normalization removes any constant scaling, using relative term frequency or raw term counts gives the same normalized vector.

3.5 Classification using Logistic Regression

Given a feature vector x with M components, the logit score z is a weighted sum of the features, and the probability that the review is synthetic (y = 1) is obtained with the sigmoid function:

$$z = w^T x + b = \Sigma_{j=1}^{M}\, w_j x_j + b$$

$$P(Y = 1 \mid x) = \sigma(z) = 1 \,/\, (1 + e^{-z})$$

The parameters w and b are obtained by minimizing the L2-regularized binary cross-entropy (log-loss) objective, in the form used by the scikit-learn implementation [6]:

$$J(w, b) = \tfrac{1}{2}\|w\|_2^2 + C \sum_{i=1}^{N} [\, -y^{(i)} \ln \sigma(z^{(i)}) - (1 - y^{(i)}) \ln(1 - \sigma(z^{(i)})) \,]$$

Here C = 1.0 is the inverse regularization strength: a smaller C penalizes large weights more strongly, which prevents parameter explosion and overfitting on the 50,000-dimensional feature space. The sigmoid output is clamped to a bounded range before thresholding.

3.6 Three-Tier Decision Calibration

Standard classifiers apply a single decision boundary at P = 0.50, which forces a confident label even for reviews that the model finds ambiguous. ReviewGuard instead defines an explicit uncertainty margin:

$$Decision(P) = GENUINE, \ \text{if } P < 0.45$$

$$Decision(P) = MEDIUM \,/\, UNCERTAIN, \ \text{if } 0.45 \leq P \leq 0.55$$

$$Decision(P) = FAKE \,/\, SUSPICIOUS, \ \text{if } P > 0.55$$

Reviews in the uncertain band are shown with an amber badge and are intended for manual administrative audit, reducing the risk of wrongly accusing genuine customers.

3.7 Evaluation Plan

Four supervised models were trained and tested on the same 80/20 stratified split of the 40,420 reviews, giving 32,336 training and 8,084 test samples: Multinomial Naive Bayes, Linear Support Vector Classifier, Random Forest (100 trees) and Logistic Regression, all using TF-IDF features. Models are compared using accuracy, precision, recall, F1-score, ROC-AUC, inference time per review and serialized model size. Training and evaluation follow standard supervised-learning practice [6], [8]. Error analysis uses the confusion matrix, treating a genuine review labelled fake as a False Positive (Type I error) and a fake review labelled genuine as a False Negative (Type II error).

CHAPTER 4

System Architecture

Technology Stack

REST API

Data Flow

User Interface

Cloud Deployment and CI/CD

Implementation Status

CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture

ReviewGuard uses a decoupled microservice architecture. The browser-based client sends the review text as JSON to a Flask application served by Gunicorn. The application loads the serialized TF-IDF vectorizer and Logistic Regression model, applies clean_text, vectorizes the text, computes the probability and returns the classification to the client, which displays it as a coloured badge. The service is hosted on Render and redeployed automatically when code is pushed to the main branch.

Figure 4.1: System architecture and data flow

4.2 Technology Stack

Table 4.1: Technology stack

Component	Role in ReviewGuard
Python / scikit-learn	Text vectorization, model training and inference [6]
Flask 3.1.2	WSGI application layer and REST endpoints [7]
Jinja2	Template engine with dynamic static-asset binding (url_for)
HTML5, CSS3, ES6 JavaScript	Semantic markup, responsive layout, dark/light mode via CSS custom properties, client logic
Browser Fetch API	Asynchronous JSON requests to /api/predict without page refresh
Gunicorn 23.0.0	Production WSGI server with multiple worker threads
Render with render.yaml	Cloud hosting configured as infrastructure as code
Git (main branch)	Version control and trigger for automated deployment

4.3 REST API

The service exposes two API endpoints. The /api/predict endpoint accepts a JSON payload containing the review text and returns the classification result for that review. The /api/health endpoint reports whether the service is running and is used to monitor the deployment.

4.4 Data Flow

The data flow is: (1) the user enters or pastes a review in the web interface; (2) the JavaScript client sends it to /api/predict using the Fetch API; (3) the server applies the five-step clean_text sanitization; (4) the TF-IDF vectorizer produces a sparse 50,000-dimensional vector; (5) Logistic Regression computes the logit score and the sigmoid

gives the probability; (6) the three-tier engine assigns the Genuine, Uncertain or Fake/Suspicious class; and (7) the client displays the result with the matching green, amber or red badge.

4.5 User Interface

The interface is built with semantic HTML5, vanilla CSS3 and ES6 JavaScript, without a front-end framework. It supports dark and light themes through CSS custom properties and updates the result in place because requests are asynchronous. Figures 4.2 and 4.3 show the home screen and a prediction result.

Figure 4.2: ReviewGuard web interface – home screen

Figure 4.3: ReviewGuard web interface – prediction result

4.6 Cloud Deployment and CI/CD

The application is continuously deployed to Render through Git integration. The deployment is described declaratively in a render.yaml blueprint. Gunicorn 23.0.0 serves the Flask application with multiple worker threads. A push to the main branch automatically triggers the dependency build from requirements.txt and a zero-downtime rolling redeployment.

4.7 Implementation Status

The text-sanitization pipeline, TF-IDF vectorizer, Logistic Regression classifier, three-tier decision engine, REST API, web interface and cloud deployment are implemented. Reviewer-account metadata, explainability features and graph-based analysis are not part of the current implementation (Chapter 6).

CHAPTER 5

Dataset Characterization

Experimental Setup

Model Comparison

Confusion Matrix Analysis

Error Analysis

Architectural Justification

Current Limitations

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Dataset Characterization

The 40,420 valid reviews span ten retail domains, with no category contributing more than about 12% of the data (Table 5.1, Figure 5.1). This spread prevents the model from memorizing the vocabulary of a single category.

Table 5.1: Category-wise distribution of reviews

Rank	Category	Reviews	Share (%)
1	Kindle Store	4,728	11.70
2	Books	4,369	10.81
3	Pet Supplies	4,251	10.52
4	Home & Kitchen	4,056	10.03
5	Electronics	3,988	9.87
6	Sports & Outdoors	3,944	9.76
7	Tools & Home Imp.	3,858	9.54
8	Clothing & Shoes	3,847	9.52
9	Toys & Games	3,792	9.38
10	Movies & TV	3,587	8.87

Figure 5.1: Category distribution of the 40,420 reviews

5.2 Experimental Setup

All four classifiers were evaluated on identical 80/20 stratified splits of the 40,420-review dataset, giving 32,336 training and 8,084 test samples. Stratification preserves the near-equal OR/CG class balance in both subsets.

5.3 Model Comparison

Table 5.2: Quantitative comparison of classifiers

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	ROC-AUC	Inference time	Disk size
Multinomial Naive Bayes	85.34	86.10	84.20	85.14	0.921	< 15 ms	1.8 MB
Linear SVC	88.95	89.80	87.90	88.84	N/A*	~ 22 ms	2.1 MB
Random Forest (n=100)	84.12	85.20	82.50	83.83	0.915	~ 210 ms	148.0 MB
Logistic Regression + TF-IDF (ReviewGuard)	89.19	89.34	88.99	89.16	0.954	~ 24 ms	2.8 MB

*LinearSVC computes geometric margins rather than native probabilities, so ROC-AUC is not standard without Platt scaling. ReviewGuard accuracy, precision, recall and F1 are computed from the confusion matrix in Figure 5.3.

Figure 5.2: Model comparison – accuracy, F1-score and ROC-AUC

Logistic Regression achieved the highest accuracy, F1-score and ROC-AUC among the compared models. LinearSVC was close in accuracy (88.95%) but does not provide native probabilities, which the three-tier calibration rule requires. Random Forest was the slowest and largest model and also the least accurate.

5.4 Confusion Matrix Analysis

Figure 5.3 shows the predictions of ReviewGuard on the 8,084 test reviews. Of 4,043 genuine reviews, 3,614 were correctly identified and 429 were labelled fake; of 4,041 fake reviews, 3,596 were detected and 445 were labelled genuine.

Figure 5.3: Confusion matrix of ReviewGuard on the test set

Table 5.3: Confusion-matrix-derived metrics for ReviewGuard

Metric	Value
Accuracy = (3,614 + 3,596) / 8,084	89.19%
Precision (Fake class) = 3,596 / (3,596 + 429)	89.34%
Recall / Sensitivity (Fake class) = 3,596 / (3,596 + 445)	88.99%
Specificity (Genuine class) = 3,614 / (3,614 + 429)	89.39%
F1-score (Fake class)	89.16%
False Positive rate = 429 / 4,043	10.61%
False Negative rate = 445 / 4,041	11.01%

5.5 Error Analysis

In consumer-trust applications the two errors carry different costs. A False Positive (a genuine review labelled fake) alienates legitimate customers and damages seller credibility. A False Negative (a deceptive review labelled genuine) is undesirable, but an isolated spam review causes less institutional damage than accusing real shoppers of fraud. The uncertainty margin of 0.45 to 0.55 addresses this asymmetry by routing borderline reviews to a manual administrative audit queue instead of forcing a verdict; the team's evaluation reported a 6.8% reduction in False Positive misclassifications on borderline reviews.

5.6 Architectural Justification

Logistic Regression gave the best F1-score (89.16%) and ROC-AUC (0.954) while preserving an inference time of about 24 ms on a basic CPU, within the 50 ms requirement, and a serialized size of 2.8 MB. By contrast, the Random Forest model is about 148 MB and roughly nine times slower (about 210 ms per prediction). These properties make Logistic Regression well suited to a low-cost microservice deployment.

5.7 Current Limitations

1. **Absence of reviewer-account metadata.** The system classifies review text in isolation. It has no access to account age, verified-purchase badges, review frequency or IP clusters.
2. **Context blindness and sarcasm.** Frequency-based n-gram features cannot model semantic irony, for example a sentence that praises a phone because it doubles as a space heater.
3. **Susceptibility to advanced LLM evasion.** Modern large language models prompted to imitate casual human writing, including natural typos, can lower the certainty of an n-gram classifier.
4. **Single-dataset evaluation.** Results are measured on one corpus of original and computer-generated reviews; performance on reviews from other platforms or newer generators has not been tested.

CHAPTER 6

Conclusion

Future Scope

Expected Impact and Responsible Use

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project developed ReviewGuard, a text-based system for detecting fake product reviews. A five-step sanitization pipeline, a 50,000-dimensional TF-IDF representation of unigrams and bigrams and an L2-regularized Logistic Regression classifier were trained on 40,420 reviews from ten product categories. On an 80/20 stratified test split the model achieved 89.19% accuracy, an F1-score of 89.16% and an ROC-AUC of 0.954, ahead of Multinomial Naive Bayes, LinearSVC and Random Forest, while using only 2.8 MB of storage and about 24 ms per prediction on a CPU. The three-tier calibration rule exposes uncertainty instead of forcing a binary verdict, and the model is served through a Flask REST service with a responsive web interface deployed on Render. The system should be regarded as a decision-support tool, because it is limited to text and to the dataset on which it was evaluated.

6.2 Future Scope

Future work includes: (i) benchmarking and integrating fine-tuned lightweight transformer models such as DeBERTa-v3 and RoBERTa for bidirectional contextual semantics; (ii) constructing bipartite reviewer–product graphs and applying Graph Convolutional Networks to detect collusive bot rings from behavioural topology [5]; (iii) integrating SHAP and LIME so that influential positive and negative tokens are highlighted in the interface; (iv) packaging ReviewGuard as a Manifest V3 browser extension that shows real-time trust badges on e-commerce product pages such as Amazon and Flipkart; and (v) adding reviewer-account metadata features and evaluating the system on reviews from additional platforms and newer language-model generators.

6.3 Expected Impact and Responsible Use

If extended and validated on platform data, ReviewGuard could help marketplaces and consumers identify suspicious reviews at low computational cost. Because a wrong accusation harms genuine customers and sellers, the tool should present probabilities and the uncertainty band clearly, keep a human in the loop for borderline cases, and avoid presenting its output as proof of fraud.

REFERENCES / BIBLIOGRAPHY

1. J. Salminen, C. Kandpal, A. M. Kamel, S. Jung, and B. J. Jansen, “Creating and detecting fake reviews of online products,” Journal of Retailing and Consumer Services, vol. 64, Art. no. 102771, 2022, doi: 10.1016/j.jretconser.2021.102771.
2. P. Hájek and A. Barushka, “A comparative study of machine learning methods for detection of fake online consumer reviews,” in Proc. 3rd Int. Conf. E-Business and Internet (ICEBI), ACM, 2019, pp. 18–22.
3. Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, “Enhancing graph neural network-based fraud detectors against camouflaged fraudsters,” in Proc. 29th ACM Int. Conf. Information and Knowledge Management (CIKM), 2020, pp. 315–324, doi: 10.1145/3340531.3411903.
4. M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2018.
5. E. Frank, M. A. Hall, and I. H. Witten, “The WEKA Workbench,” online appendix for Data Mining: Practical Machine Learning Tools and Techniques, 4th ed. Morgan Kaufmann, 2016.
6. M. Luca, “Reviews, reputation, and revenue: The case of Yelp.com,” Harvard Business School Working Paper 12-016, 2011.

Approved by:

Project Guide

Prof. Bharti Kungwani

Prof. Shubhangi Tirpude

Associate Head

Dr. Pravin Sonsare

Dean SoCSE

Dr. P. D. Adane

ANNEXURE A: PUBLICATIONS AND INTELLECTUAL PROPERTY

At the time of preparing this report, no publication, patent or registered intellectual-property output was identified in the supplied project materials.

ANNEXURE B: STUDENT CONTRIBUTION STATEMENT

The specific and verifiable technical contribution of each team member is listed below.

Student Name	Roll Number	Contribution
Prashant Raghuwanshi	A4-27	TF-IDF feature extraction (unigrams and bigrams, 50,000 features), training and comparison of Multinomial Naive Bayes, LinearSVC, Random Forest and Logistic Regression, three-tier threshold calibration, and performance evaluation (confusion matrix, ROC-AUC, error analysis).
Swyam Gupta	A4-28	Flask REST API (/api/predict, /api/health), HTML5/CSS3/JavaScript user interface with dark/light mode, Gunicorn and Render deployment with render.yaml and Git-based CI/CD, plus report documentation and testing.
Kush Jajoo	A4-30	Dataset collection and study of the 40,432-review corpus, text sanitization pipeline (clean_text), duplicate removal and class-balance verification. Literature review of the five reference papers and compilation of the comparative literature matrix.

ANNEXURE C: PROJECT OUTPUT

The major technological and academic outputs delivered by the project include:

1. Working Prototype: AI-based fake product review detection that classifies a review as Genuine, Uncertain or Fake/Suspicious.

2. Software/Application: Flask REST microservice (/api/predict and /api/health) with a responsive HTML5/CSS3/JavaScript interface, deployed on Render.

3. Dataset Pipeline: Preprocessing of a 40,432-review, ten-category corpus of original and computer-generated reviews into 40,420 cleaned samples.

4. Text Processing Pipeline: Five-step deterministic clean_text sanitization followed by TF-IDF (1,2)-gram vectorization with 50,000 features.

5. Trained Model: L2-regularized Logistic Regression classifier (2.8 MB, about 24 ms per prediction on a CPU) with a three-tier calibration engine.

6. Deployment Configuration: render.yaml blueprint, Gunicorn 23.0.0 production server and automated Git-based CI/CD.

7. GitHub or Project Repository: https://github.com/24jajook1-sys/Fake-review-ML

8. Live Demonstration: https://reviewguard-app.onrender.com/